Smart HAS

Monitoramento de Hipertensão Arterial Sistêmica

Fase 7 — Consolidação em Oracle PL/SQL e Automação CI/CD

Documentação técnica da entrega

Integrantes do grupo

Gabriel Garuti Paiva Cracco — RM 554866

Vitor Cremonezi Morija — RM 550998

Repositório: https://github.com/GabrielGaruti/smarthas

Vídeo: [inserir link do vídeo no YouTube]

FIAP · Novembro de 2026

1. Visão geral da Fase 7

O Smart HAS é uma solução de monitoramento da Hipertensão Arterial Sistêmica: o paciente registra suas leituras de pressão pelo app mobile (React Native), a API central (Java/Spring Boot) classifica cada leitura e aciona as regras clínicas no banco Oracle, e a equipe de saúde acompanha indicadores, relatórios e alertas pelo painel Angular.

Nesta fase o foco foi profissionalizar a entrega. Na Parte 1, a camada PL/SQL criada na Fase 6 foi consolidada em packages, ganhou triggers de auditoria e automação e passou a usar SQL dinâmico para consultas flexíveis em tempo real. Na Parte 2, foi criado um pipeline CI/CD no GitHub Actions que compila, testa, instala o banco Oracle, roda testes de integração, publica a imagem Docker da API, faz o deploy em um ambiente de staging com Oracle e gera logs e relatórios de cada execução.

Parte	Entrega	Onde está no repositório
1 — Packages	4 packages (spec + body) substituindo as 8 rotinas avulsas da Fase 6	database/oracle/02 a 05_pkg_shas_*.sql
1 — Triggers	3 triggers (auditoria de alertas, status automático, validação de dispositivo IoT) + tabela de auditoria	database/oracle/06_triggers.sql, 01_ddl_tabelas.sql
1 — Dynamic SQL	Busca com filtros combináveis e GROUP BY por dimensão escolhida em tempo real	PKG_SHAS_INDICADORES (03_pkg_shas_indicadores.sql)
1 — Integração	Gateway Java chamando os packages; 4 novos endpoints administrativos	backend-springboot/.../integration, web/AdminController.java
2 — CI/CD	Workflow com 4 jobs: backend, web, oracle-integration e deploy	.github/workflows/ci-cd.yml
2 — Testes	28 testes automatizados (H2) + 8 testes de integração com Oracle real	backend-springboot/src/test
2 — Deploy	Dockerfile, imagem no GHCR, scripts de migração e smoke test pós-deploy	backend-springboot/Dockerfile, ci/, database/oracle/run_all.sql e deploy_plsql.sql

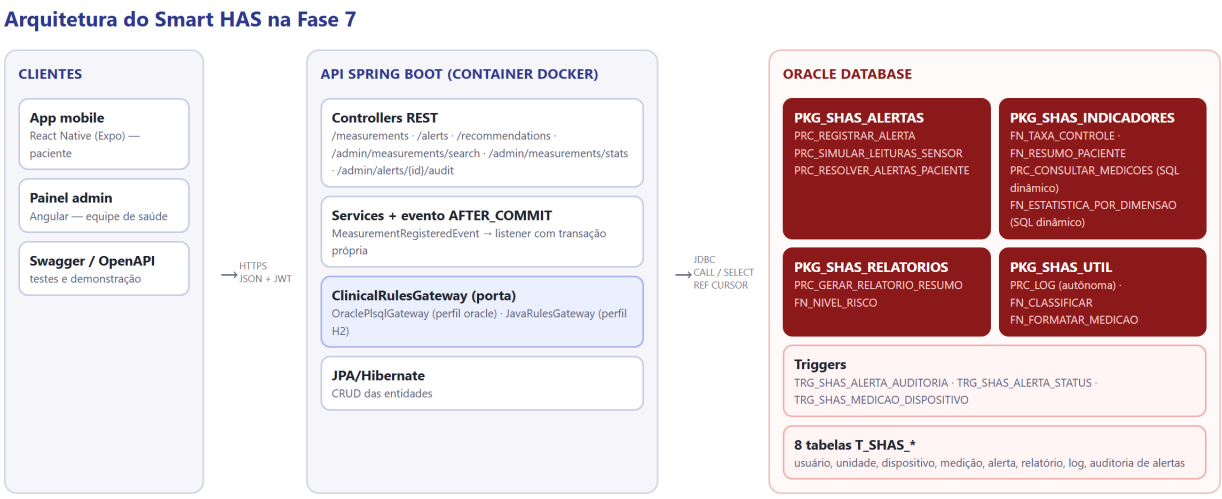


Figura 1 — Arquitetura do Smart HAS na Fase 7

2. Parte 1 — Consolidação em Oracle PL/SQL

2.1 Organização em packages

Na Fase 6 havia oito rotinas avulsas (PRC_SHAS_* e FN_SHAS_*). Elas foram agrupadas por responsabilidade em quatro packages. Cada package tem uma especificação (spec), que é o contrato público usado pelo Java e pelos outros packages, e um corpo (body), com a implementação e as rotinas privadas.

Package	Membros públicos	Responsabilidade
PKG_SHAS_UTIL	PRC_LOG, FN_CLASSIFICAR, FN_FORMATAR_MEDICAO; constantes c_info/c_aviso/c_erro e meta 140/90	Apoio usado por todos os objetos: log autônomo, regra de classificação, formatação
PKG_SHAS_INDICADORES	FN_TAXA_CONTROLE, FN_RESUMO_PACIENTE, PRC_CONSULTAR_MEDICOES, FN_ESTATISTICA_POR_DIMENSAO	Indicadores do app e do painel e as consultas com SQL dinâmico
PKG_SHAS_ALERTAS	PRC_REGISTRAR_ALERTA, PRC_SIMULAR_LEITURAS_SENSOR, PRC_RESOLVER_ALERTAS_PACIENTE	Regra clínica de alertas (acionada pelo back-end), entrada de dados IoT e encerramento de alertas
PKG_SHAS_RELATORIOS	PRC_GERAR_RELATORIO_RESUMO, FN_NIVEL_RISCO	Rotina batch de consolidação por paciente, acionada pelo painel admin

Mapeamento das rotinas da Fase 6 para os packages:

Fase 6 (rotina avulsa)	Fase 7 (membro de package)
PRC_SHAS_LOG	PKG_SHAS_UTIL.PRC_LOG
FN_SHAS_CLASSIFICAR	PKG_SHAS_UTIL.FN_CLASSIFICAR
FN_SHAS_FORMATAR_MEDICAO	PKG_SHAS_UTIL.FN_FORMATAR_MEDICAO
FN_SHAS_TAXA_CONTROLE	PKG_SHAS_INDICADORES.FN_TAXA_CONTROLE
FN_SHAS_RESUMO_PACIENTE	PKG_SHAS_INDICADORES.FN_RESUMO_PACIENTE
PRC_SHAS_REGISTRAR_ALERTA	PKG_SHAS_ALERTAS.PRC_REGISTRAR_ALERTA
PRC_SHAS_SIMULAR_LEITURAS_SENSOR	PKG_SHAS_ALERTAS.PRC_SIMULAR_LEITURAS_SENSOR
PRC_SHAS_GERAR_RELATORIO_RESUMO	PKG_SHAS_RELATORIOS.PRC_GERAR_RELATORIO_RESUMO
— (nova)	PKG_SHAS_INDICADORES.PRC_CONSULTAR_MEDICOES, FN_ESTATISTICA_POR_DIMENSAO
— (nova)	PKG_SHAS_ALERTAS.PRC_RESOLVER_ALERTAS_PACIENTE, PKG_SHAS_RELATORIOS.FN_NIVEL_RISCO

Ganhos da organização em packages:

- Encapsulamento: rotinas internas, como validar_dias em PKG_SHAS_INDICADORES, ficam só no body e não podem ser chamadas de fora.
- Reuso e um único ponto de verdade: a meta terapêutica (140/90) virou constante em PKG_SHAS_UTIL e é usada pela classificação, pelo indicador, pela regra de alerta e pelo SQL dinâmico.
- Menos invalidação em cascata: mudar o body não invalida quem depende só da spec, então ajustes de implementação não derrubam as outras rotinas nem a aplicação.

- Desempenho: o package inteiro é carregado na memória na primeira chamada, e as chamadas seguintes da sessão são mais rápidas.
- Organização e permissões: o Java chama PKG_SHAS_ALERTAS.PRC_REGISTRAR_ALERTA usando o package como "catalog" do JDBC, e um GRANT EXECUTE por package basta para liberar um conjunto de rotinas.

```
CREATE OR REPLACE PACKAGE PKG_SHAS_UTIL AS
  c_meta_sistolica  CONSTANT PLS_INTEGER := 140;
  c_meta_diastolica CONSTANT PLS_INTEGER := 90;
  PROCEDURE PRC_LOG (p_rotina IN VARCHAR2, p_evento IN VARCHAR2, p_mensagem IN VARCHAR2);
  FUNCTION  FN_CLASSIFICAR (p_sistolica IN NUMBER, p_diastolica IN NUMBER,
                            p_formato IN VARCHAR2 DEFAULT 'CODIGO') RETURN VARCHAR2 DETERMINISTIC;
  FUNCTION  FN_FORMATAR_MEDICAO (p_id_medicao IN NUMBER) RETURN VARCHAR2;
END PKG_SHAS_UTIL;
```

2.2 Triggers de auditoria e automação

Trigger	Evento	Tipo	O que faz e valor agregado
TRG_SHAS_ALERTA_AUDITORIA	AFTER INSERT OR UPDATE OR DELETE em T_SHAS_ALERTA, FOR EACH ROW	Auditoria	Grava em T_SHAS_AUDITORIA_ALERTA a operação, o status anterior e o novo, a severidade, o usuário do banco e o programa de origem (ex.: "JDBC Thin Client" = API Java). Cada alerta CRÍTICO criado também vira evento no log. Dá rastreabilidade clínica: quem criou, alterou ou encerrou cada alerta, e quando.
TRG_SHAS_ALERTA_STATUS	BEFORE UPDATE OF ST_ALERTA em T_SHAS_ALERTA, FOR EACH ROW	Automação	Preenche DT_RESOLUCAO quando o alerta passa de ABERTO para RESOLVIDO e limpa a data se ele for reaberto. Quem resolve (app, painel, procedure ou SQL manual) só muda o status; a data fica sempre consistente.
TRG_SHAS_MEDICAO_DISPOSITIVO	BEFORE INSERT em T_SHAS_MEDICAO, FOR EACH ROW WHEN (ID_DISPOSITIVO não nulo)	Automação e integridade	Só aceita leitura de aparelho ativo e do próprio paciente (erros -20030/-20031/-20032), marca a origem como SENSOR e atualiza DT_ULTIMA_SYNC do aparelho. Protege o banco de dados IoT inconsistentes, venham de onde vierem.

Por que o alerta não é gerado por trigger em T_SHAS_MEDICAO? A regra de tendência precisa ler as últimas leituras da própria T_SHAS_MEDICAO, o que numa trigger de linha causaria o erro ORA-04091 (tabela mutante). Por isso o alerta continua sendo disparado pelo evento de back-end (AFTER_COMMIT), que chama PKG_SHAS_ALERTAS.PRC_REGISTRAR_ALERTA. As triggers ficaram com o que é seguro e natural fazer no nível da linha: auditar, manter datas e validar a integridade.

```
CREATE OR REPLACE TRIGGER TRG_SHAS_ALERTA_AUDITORIA
AFTER INSERT OR UPDATE OR DELETE ON T_SHAS_ALERTA
FOR EACH ROW
DECLARE
  v_operacao T_SHAS_AUDITORIA_ALERTA.TP_OPERACAO%TYPE;
BEGIN
  v_operacao := CASE WHEN INSERTING THEN 'INSERT' WHEN UPDATING THEN 'UPDATE' ELSE 'DELETE' END;
  INSERT INTO T_SHAS_AUDITORIA_ALERTA (ID_ALERTA, ID_USUARIO, TP_OPERACAO, ST_ANTERIOR, ST_NOVO,
                                       NV_SEVERIDADE, DS_ORIGEM)
  VALUES (NVL(:NEW.ID_ALERTA, :OLD.ID_ALERTA), NVL(:NEW.ID_USUARIO, :OLD.ID_USUARIO), v_operacao,
          :OLD.ST_ALERTA, :NEW.ST_ALERTA, NVL(:NEW.NV_SEVERIDADE, :OLD.NV_SEVERIDADE),
          SYS_CONTEXT('USERENV', 'MODULE') || ' @ ' || SYS_CONTEXT('USERENV', 'HOST'));
  IF INSERTING AND :NEW.NV_SEVERIDADE = 'CRITICO' THEN
    PKG_SHAS_UTIL.PRC_LOG('TRG_SHAS_ALERTA_AUDITORIA', PKG_SHAS_UTIL.c_aviso, 'Alerta CRITICO ...');
  END IF;
END;
```

Nova tabela T_SHAS_AUDITORIA_ALERTA (ID_AUDITORIA, ID_ALERTA, ID_USUARIO, TP_OPERACAO, ST_ANTERIOR, ST_NOVO, NV_SEVERIDADE, NM_USUARIO_BD, DS_ORIGEM, DT_OPERACAO). Ela não tem FK para T_SHAS_ALERTA de propósito: o histórico sobrevive mesmo se o alerta for excluído (por exemplo, pelo ON DELETE CASCADE da medição).

2.3 Dynamic SQL para consultas em tempo real

O painel administrativo precisa combinar filtros livremente: paciente, classificação, origem (manual ou sensor), período e sistólica mínima, com ordenação e limite escolhidos pelo usuário. Com SQL estático seria preciso uma consulta para cada combinação, ou uma única consulta cheia de condições "OR parâmetro IS NULL", que o otimizador trata mal. A procedure PKG_SHAS_INDICADORES.PRC_CONSULTAR_MEDICOES monta o comando em tempo de execução só com os filtros informados.

- EXECUTE IMMEDIATE ... INTO ... USING calcula o total de registros sem o limite.
- OPEN cursor FOR ... USING devolve as linhas num SYS_REFCURSOR, lido pelo Java como ResultSet.
- Binds de quantidade fixa: um filtro ausente vira "(1 = 1 OR :b IS NULL)", que o otimizador elimina, e a lista do USING fica sempre igual.
- Segurança contra SQL injection: os valores entram sempre como bind variables; o ORDER BY e o GROUP BY só aceitam nomes de uma lista fechada (whitelist) definida no package. Um valor fora da lista gera ORA-20021.
- Transparência: a procedure devolve em p_sql o comando gerado, que aparece na resposta da API e no log de processamento.

```
add_filtro(p_id_usuario IS NOT NULL,  'm.ID_USUARIO = :b_usuario', ':b_usuario');
add_filtro(v_classif IS NOT NULL,
           'PKG_SHAS_UTIL.FN_CLASSIFICAR(m.VL_SISTOLICA, m.VL_DIASTOLICA) = :b_classif', ':b_classif');
add_filtro(v_origem IS NOT NULL,      'm.TP_ORIGEM = :b_origem', ':b_origem');
...
v_order := CASE UPPER(p_ordenar_por) WHEN 'DATA' THEN 'm.DT_MEDICAO' WHEN 'SISTOLICA' THEN 'm.VL_SISTOLICA'
                                     WHEN 'DIASTOLICA' THEN 'm.VL_DIASTOLICA' WHEN 'PACIENTE' THEN
'u.NM_COMPLETO' END;

EXECUTE IMMEDIATE 'SELECT COUNT(*)' || v_from || v_where INTO p_total
  USING p_id_usuario, v_classif, v_origem, p_dias, p_sistolica_min;

p_sql := v_select || v_from || v_where || v_order || ' FETCH FIRST :b_limite ROWS ONLY';
OPEN p_cursor FOR p_sql USING p_id_usuario, v_classif, v_origem, p_dias, p_sistolica_min, p_limite;
```

Exemplo real do comando gerado para "hipertensão, ordenado pela maior sistólica, 5 linhas" (capturado no smoke test do pipeline):

```
SELECT m.ID_MEDICAO, m.ID_USUARIO, u.NM_COMPLETO, m.VL_SISTOLICA, m.VL_DIASTOLICA, m.VL_FREQ_CARDIACA,
m.DT_MEDICAO, m.TP_ORIGEM, PKG_SHAS_UTIL.FN_CLASSIFICAR(m.VL_SISTOLICA, m.VL_DIASTOLICA) AS
CD_CLASSIFICACAO
  FROM T_SHAS_MEDICAO m JOIN T_SHAS_USUARIO u ON u.ID_USUARIO = m.ID_USUARIO
 WHERE 1 = 1
   AND (1 = 1 OR :b_usuario IS NULL)
   AND PKG_SHAS_UTIL.FN_CLASSIFICAR(m.VL_SISTOLICA, m.VL_DIASTOLICA) = :b_classif
   AND (1 = 1 OR :b_origem IS NULL)
   AND (1 = 1 OR :b_dias IS NULL)
   AND (1 = 1 OR :b_sis_min IS NULL)
 ORDER BY m.VL_SISTOLICA DESC, m.ID_MEDICAO DESC
 FETCH FIRST :b_limite ROWS ONLY
```

A function FN_ESTATISTICA_POR_DIMENSAO usa a mesma técnica para o GROUP BY: a dimensão (CLASSIFICACAO, ORIGEM, PACIENTE ou PERIODO_DIA) é escolhida em tempo real e a função devolve quantidade, médias e % na meta de cada grupo. As duas rotinas são expostas na API em GET /admin/measurements/search e GET /admin/measurements/stats.

2.4 Documentação dos objetos criados

Todos os objetos do banco ao fim da Fase 7, com finalidade e valor agregado:

Objeto	Tipo	Finalidade	Valor agregado
PKG_SHAS_UTIL.PRC_LOG	procedure (autônoma)	Grava eventos em T_SHAS_LOG_PROCESSAMENTO	Log persiste mesmo com ROLLBACK; pode ser chamado de functions e triggers
PKG_SHAS_UTIL.FN_CLASSIFICAR	function DETERMINISTIC	Normal / Elevada / Hipertensão	Mesma regra do Java; usável em SELECT, WHERE e GROUP BY
PKG_SHAS_UTIL.FN_FORMATAR_MEDICAO	function	Linha legível de uma leitura	Dados prontos para exibição e relatórios
PKG_SHAS_INDICADORES.FN_TAXA_CONTROLE	function	% de leituras na meta em N dias	Indicador de adesão no app e no painel
PKG_SHAS_INDICADORES.FN_RESUMO_PACIENTE	function	Texto-resumo do paciente	Reaproveitado pelo app, painel e relatório
PKG_SHAS_INDICADORES.PRC_CONSULTAR_MEDICOES	procedure (SQL dinâmico)	Busca com filtros, ordenação e limite em tempo real	Painel flexível sem SQL fixo por combinação, protegido contra injection
PKG_SHAS_INDICADORES.FN_ESTATISTICA_POR_DIMENSAO	function (SQL dinâmico)	GROUP BY pela dimensão escolhida	Análises do painel com uma única rotina
PKG_SHAS_ALERTAS.PRC_REGISTRAR_ALERTA	procedure	Avalia a leitura e grava alerta (crise, tendência, hipertensão)	Alerta no momento da medição; idempotente
PKG_SHAS_ALERTAS.PRC_SIMULAR_LEITURAS_SENSOR	procedure	Importa leituras simuladas de IoT	Prepara o sistema para monitores conectados
PKG_SHAS_ALERTAS.PRC_RESOLVER_ALERTAS_PACIENTE	procedure	Encerra os alertas abertos do paciente	Fluxo de pós-consulta em uma chamada; triggers cuidam de data e auditoria
PKG_SHAS_RELATORIOS.PRC_GERAR_RELATORIO_RESUMO	procedure (batch)	Consolida indicadores por paciente	Relatório pronto no banco, com tratamento de erro por paciente

Objeto	Tipo	Finalidade	Valor agregado
PKG_SHAS_RELATORIOS.FN_NIVEL_RISCO	function DETERMINISTIC	SEM DADOS / BAIXO / MODERADO / ALTO	Regra de risco isolada e reutilizável
TRG_SHAS_ALERTA_AUDITORIA	trigger	Auditoria de alertas	Rastreabilidade clínica
TRG_SHAS_ALERTA_STATUS	trigger	Data de resolução automática	Consistência sem depender de quem altera
TRG_SHAS_MEDICAO_DISPOSITIVO	trigger	Validação de aparelho IoT e sincronização	Integridade dos dados de sensores
T_SHAS_AUDITORIA_ALERTA	tabela	Histórico das operações em alertas	Base para auditoria e compliance (LGPD)
run_all.sql / deploy_plsql.sql / 99_validar_objetos.sql	scripts	Instalação completa, deploy incremental e validação	Automatizam o banco no pipeline

2.5 Integração com o back-end Java

A porta ClinicalRulesGateway ganhou quatro métodos (busca dinâmica, estatística por dimensão, resolução em lote e auditoria). No perfil oracle, o OraclePlsqlGateway chama os packages com SimpleJdbcCall usando o nome do package como catalog, lê os REF CURSORs como ResultSet e chama a function que devolve cursor com a sintaxe JDBC {? = call ...}. No perfil H2, o JavaRulesGateway implementa as mesmas regras e validações em Java, para desenvolvimento e testes sem Oracle.

```
searchCall = new SimpleJdbcCall(jdbc)
        .withCatalogName("PKG_SHAS_INDICADORES")
        .withProcedureName("PRC_CONSULTAR_MEDICOES")
        .withoutProcedureColumnMetaDataAccess()
        .declareParameters(
                new SqlParameter("P_ID_USUARIO", Types.NUMERIC),
                ...
                new SqlOutParameter("P_TOTAL", Types.NUMERIC),
                new SqlOutParameter("P_SQL", Types.VARCHAR),
                new SqlOutParameter("P_CURSOR", ORACLE_CURSOR, SEARCH_ROW));   // REF CURSOR -> List<Row>
```

Endpoint (perfil ADMIN)	Rotina Oracle
GET /admin/measurements/search	PKG_SHAS_INDICADORES.PRC_CONSULTAR_MEDICOES (SQL dinâmico)
GET /admin/measurements/stats?dimension=	PKG_SHAS_INDICADORES.FN_ESTATISTICA_POR_DIMENSAO (GROUP BY dinâmico)
POST /admin/patients/{id}/alerts/resolve	PKG_SHAS_ALERTAS.PRC_RESOLVER_ALERTAS_PACIENTE + triggers de status e auditoria
GET /admin/alerts/{id}/audit	T_SHAS_AUDITORIA_ALERTA (gravada por TRG_SHAS_ALERTA_AUDITORIA)
POST /measurements (USER)	PKG_SHAS_ALERTAS.PRC_REGISTRAR_ALERTA (evento AFTER_COMMIT)
POST /admin/reports/summary	PKG_SHAS_RELATORIOS.PRC_GERAR_RELATORIO_RESUMO

Novos códigos de erro: -20020 (filtro inválido), -20021 (ordenação ou dimensão inválida), -20022 (limite fora de 1 a 500), -20030 a -20032 (aparelho de outro paciente, inativo ou inexistente) e -20099 (deploy com objetos inválidos). Como na Fase 6, o GlobalExceptionHandler transforma ORA-20xxx em HTTP 400 com a mensagem da regra.

2.6 Correção encontrada pelos testes automatizados

Ao escrever os testes da regra de classificação, o caso 141x85 falhou: o sistema o classificava como "Elevada". A regra avaliava a faixa de "Elevada" (sistólica 120–139 ou diastólica 80–89) antes da faixa de hipertensão, então uma diastólica de 85 "ganhava" de uma sistólica de 141. O erro existia igual no Java e no PL/SQL. A ordem foi invertida nas duas camadas — hipertensão (≥ 140 ou ≥ 90) é avaliada primeiro — e o caso virou teste permanente. É um exemplo concreto do valor do pipeline: o defeito estava no sistema desde as primeiras fases e só apareceu com testes automatizados.

3. Parte 2 — Automação e Entrega Contínua

3.1 Visão geral do pipeline

O pipeline foi feito no GitHub Actions, porque o código já está no GitHub: não exige servidor próprio (como o Jenkins), o histórico de execuções fica junto do código e os containers de serviço permitem subir um Oracle real a cada execução. O arquivo é .github/workflows/ci-cd.yml.

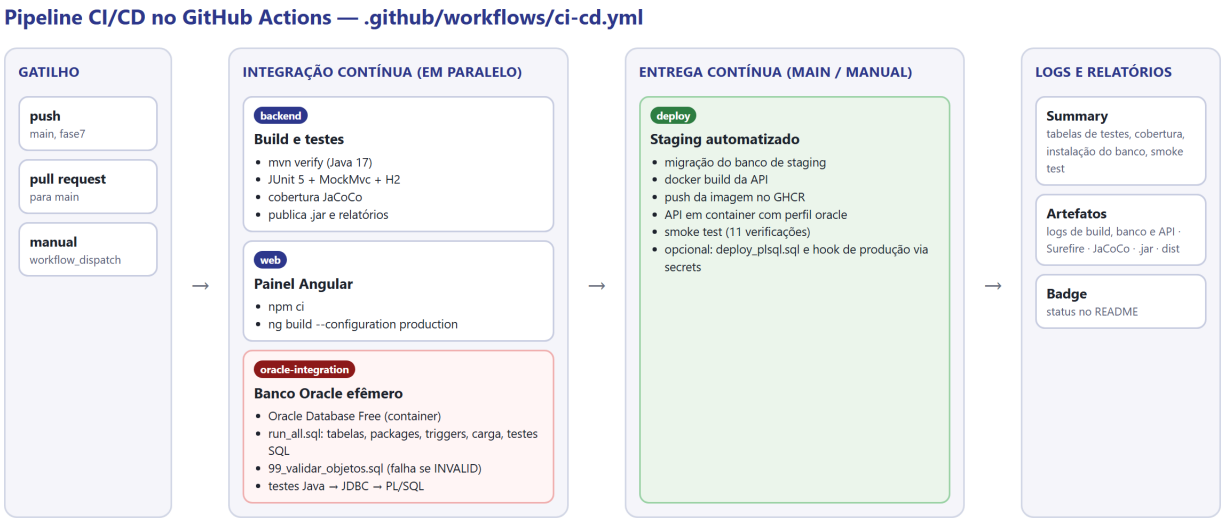


Figura 2 — Jobs do pipeline CI/CD

Job	Quando roda	Etapas
backend	Todo push (main, fase7), pull request e disparo manual	Checkout → Java 17 com cache Maven → mvn verify (compilação, testes, JaCoCo) → resumo → artefatos
web	Idem	Node 20 com cache npm → npm ci → ng build --configuration production → artefato dist
oracle-integration	Idem	Container Oracle Database Free → run_all.sql (schema, packages, triggers, validação, carga, testes SQL) → testes de integração Java/JDBC/PL-SQL (@Tag("oracle")) → resumo → logs
deploy	Push na main ou disparo manual, só se os três jobs anteriores passarem	Banco de staging migrado → docker build → push no GHCR → API em container (perfil oracle) → smoke test → deploy opcional em produção → relatório

3.2 Build e testes automatizados

O back-end ganhou uma suíte de testes com JUnit 5, AssertJ, MockMvc e H2 em memória. Os testes de integração com Oracle ficam separados pela tag "oracle" (o Maven Surefire os exclui por padrão) e rodam no job oracle-integration contra o banco recém-instalado.

Classe	Tipo	Testes	O que verifica
ClassificationTest	unitário (parametrizado)	9	Limites da classificação, incluindo 141x85 e 130x90
JavaRulesGatewayTest	integração (Spring + H2)	11	Alerta crítico idempotente, tendência, taxa de controle por período, busca dinâmica com filtros/ordenação/limite, whitelist, estatística por dimensão, resolução em lote, relatório

Classe	Tipo	Testes	O que verifica
ApiFlowTest	ponta a ponta (MockMvc)	8	Healthcheck, JWT obrigatório, login inválido, medição crítica com alerta na resposta, validação, rota ADMIN bloqueada para paciente, erro 400 de filtro, estatística e relatório
OraclePlsqlGatewayIT	integração com Oracle real	8	Packages e triggers VALID, procedure de alerta + auditoria, trigger de data de resolução, SQL dinâmico e whitelist, function com REF CURSOR, indicadores e relatório

Resultado da execução de referência do pipeline (#4): 28 testes no H2 e 8 testes de integração com Oracle, todos aprovados; cobertura de linhas de 62,4% (JaCoCo, perfil H2 — o adaptador Oracle é coberto pelos testes de integração).

3.3 Deploy automatizado do back-end e integração com o Oracle

- Banco: o job sobe um Oracle Database Free (gvenzl/oracle-free) como container de serviço, copia os scripts e executa run_all.sql com o SQL*Plus. O script para no primeiro erro (WHENEVER SQLERROR EXIT FAILURE) e o 99_validar_objetos.sql derruba o deploy se algum package ou trigger ficar INVALID — o SQL*Plus sozinho não falha em erro de compilação.
- Imagem: o Dockerfile faz o build em duas etapas (Maven → JRE 17, usuário sem privilégios). A imagem é publicada no GitHub Container Registry com a tag do commit e latest.
- Staging: a API sobe em container com o perfil oracle, conectada ao banco recém-migrado. O pipeline espera o /health responder e roda o smoke test.
- Smoke test (ci/smoke-test.sh): 11 verificações contra a API implantada — health, OpenAPI, login ADMIN e paciente, medição crítica gerando alerta pelo package, motor ORACLE_PLSQL, SQL dinâmico, GROUP BY dinâmico, bloqueio de rota ADMIN, relatório e auditoria pela trigger.
- Produção (opcional): se os secrets ORACLE_PROD_URL, ORACLE_PROD_USER e ORACLE_PROD_PASSWORD existirem, o pipeline aplica deploy_plsql.sql no banco real (deploy incremental que não apaga dados: cria a tabela de auditoria se faltar, remove as rotinas avulsas da Fase 6, recompila packages e triggers e valida). Com o secret DEPLOY_HOOK_URL, ele também dispara o deploy da imagem num serviço de hospedagem (ex.: Render ou Railway).

3.4 Logs e relatórios de execução

- Aba Summary de cada execução: tabela de testes por classe, cobertura, resumo da instalação do banco, resultado do smoke test com o SQL gerado e dados do deploy (imagem, ambiente, produção).
- Artefatos para download (14 dias): backend-relatorios (log do build, Surefire, JaCoCo HTML, .jar), web-dist, oracle-relatorios (log da instalação do schema e dos testes de integração) e deploy-relatorios (log da migração, log da API em staging, relatório do smoke test).
- Badge de status no README e histórico de execuções na aba Actions do repositório.
- No banco, cada rotina continua registrando em T_SHAS_LOG_PROCESSAMENTO, inclusive as consultas dinâmicas.

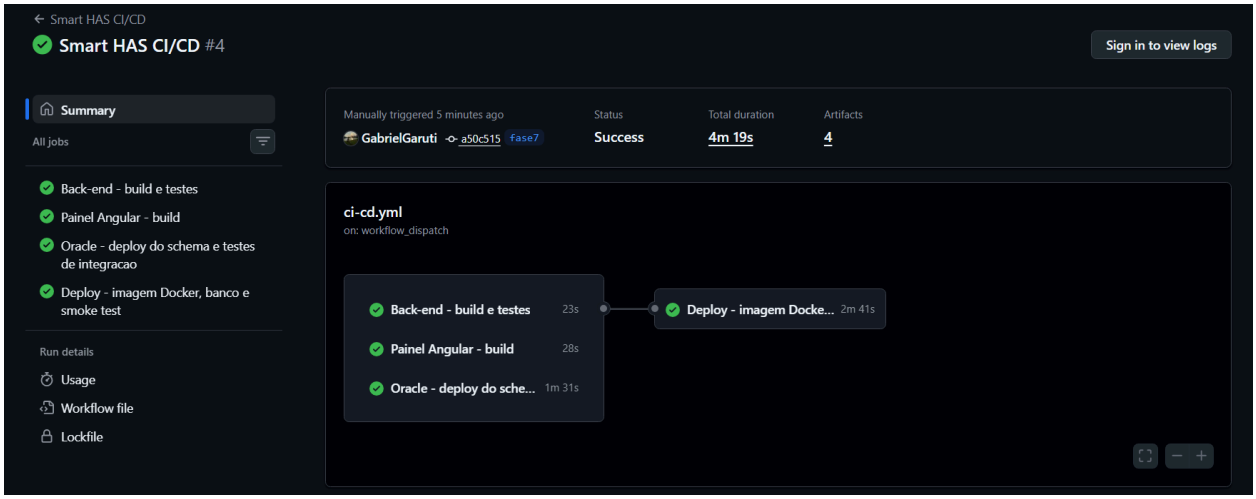


Figura 3 — Execução #4 do pipeline no GitHub Actions

Job	Resultado	Duração
backend — Back-end: build e testes	sucesso	23 s
web — Painel Angular: build	sucesso	28 s
oracle-integration — schema Oracle + 8 testes de integração	sucesso	1 min 31 s
deploy — imagem Docker, banco de staging e smoke test (11/11)	sucesso	2 min 41 s

3.5 Segurança no pipeline

- Nenhuma credencial real no código: o banco de CI/staging é efêmero (existe só durante o job) e o de produção usa secrets do GitHub, que não aparecem nos logs.
- A chave JWT de staging é gerada por execução; em produção vem de SMARTHAS_JWT_SECRET.
- Permissões mínimas: só o job de deploy tem packages: write, para publicar a imagem.
- Container da API roda com usuário sem privilégios.
- O SQL dinâmico usa binds e whitelist, e o teste OraclePlsqlGatewayIT tenta uma injeção no ORDER BY para garantir que ela é recusada.

4. Como executar

O quê	Como
Banco Oracle (do zero)	SQL Developer: abrir database/oracle/run_all.sql e executar com F5 (ou os scripts 01 a 08 em ordem). SQL*Plus: sqlplus usuario/senha@host:1521/servico @run_all.sql
Banco que já tem a Fase 6	Executar database/oracle/deploy_plsql.sql (não apaga dados)
API com Oracle	Definir ORACLE_URL, ORACLE_USER e ORACLE_PASSWORD e rodar mvn spring-boot:run -Dspring-boot.run.profiles=oracle
API sem Oracle (H2)	mvn spring-boot:run — Swagger em http://localhost:8080/swagger-ui.html
Testes	mvn verify (relatório de cobertura em target/site/jacoco/index.html)
Docker	docker build -t smarthas-api backend-springboot && docker run -p 8080:8080 smarthas-api
Pipeline	Automático a cada push; o deploy roda em push na main ou em Actions → Smart HAS CI/CD → Run workflow
Painel e app	web-angular: npm install && npm start; mobile-react-native: npm install && npx expo start

Usuários de demonstração: admin@smarthas.com / admin123 (ADMIN) e paciente@smarthas.com / 123456 (USER).

5. Conclusão e próximos passos

A Fase 7 deixou o Smart HAS mais organizado no banco e muito mais confiável na entrega. As regras clínicas estão em packages coesos, as triggers garantem auditoria e consistência independentemente de quem altera os dados, e o SQL dinâmico dá ao painel consultas flexíveis sem abrir mão da segurança. Cada mudança no código agora passa automaticamente por build, testes, instalação completa do banco Oracle, testes de integração e um deploy de staging verificado, com logs e relatórios de cada etapa — e esse processo já encontrou e ajudou a corrigir um erro de classificação que existia desde as primeiras fases.

Próximos passos: hospedar o ambiente de produção (API em nuvem com banco Oracle gerenciado, ativando os secrets já previstos no pipeline), adicionar à interface do painel a busca dinâmica e a trilha de auditoria (hoje disponíveis na API e no Swagger), agendar o relatório com DBMS_SCHEDULER e incorporar IA generativa para explicar as leituras ao paciente em linguagem simples, sempre com as regras clínicas como guarda.

Links

- Repositório GitHub: https://github.com/GabrielGaruti/smarthas
- Pipeline (GitHub Actions): https://github.com/GabrielGaruti/smarthas/actions/workflows/ci-cd.yml
- Execução de referência: https://github.com/GabrielGaruti/smarthas/actions/runs/41755711309
- Vídeo de demonstração (YouTube, não listado): [inserir link do vídeo no YouTube]